PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS
Escola de Ciências Exatas e da Computação
Mensageria e Streaming de Dados

Atividade Prática

HTTP Chunked Transfer Encoding — Streaming de Dados Progressivo

Disciplina: CIC7019 — Segurança de Sistemas e Aplicações	**Carga Horária:** 3 horas (individual ou em dupla)
Capítulo: 02 — HTTP Chunked Transfer Encoding	**Valor:** 10,0 pontos
Modalidade: Prática — Implementação e Análise	**Entrega:** Via Moodle — submeter .zip com código-fonte e relatório

1. Objetivos de Aprendizagem

- Compreender o funcionamento do protocolo HTTP Chunked Transfer Encoding (RFC 7230) em nível de wire format;
- Implementar endpoints de streaming progressivo utilizando Spring Boot (Java) ou Flask/FastAPI (Python);
- Consumir streams HTTP no lado cliente utilizando a Fetch API com ReadableStream;
- Analisar o comportamento de rede de respostas chunked com ferramentas de inspeção HTTP;
- Identificar e mitigar o problema de buffering intermediário em proxies reversos (Nginx).

2. Fundamentação Teórica Essencial

Esta seção apresenta os conceitos necessários para a realização das tarefas práticas. Recomenda-se a leitura integral antes de iniciar a implementação.

2.1 O Problema do Request-Response Convencional

No modelo HTTP convencional, o servidor deve conhecer o Content-Length da resposta antes de iniciá-la. Isso obriga a aplicação a processar ou armazenar completamente o recurso em memória

antes de qualquer transmissão ao cliente. Em cenários como exportação de relatórios de dezenas de milhares de registros, geração de tokens de linguagem natural ou transmissão de arquivos volumosos, essa abordagem apresenta três problemas críticos:

- Consumo de memória proporcional ao tamanho total do recurso no servidor;
- Latência percebida alta: o cliente aguarda o processamento completo antes de receber o primeiro byte;
- Risco de timeout: operações longas podem exceder os limites de tempo configurados em clientes e proxies.

2.2 Chunked Transfer Encoding — RFC 7230

O cabeçalho Transfer-Encoding: chunked, definido na RFC 7230, permite que o servidor inicie a transmissão de uma resposta HTTP sem conhecer seu tamanho total. O Content-Length é omitido e substituído por um protocolo de delimitação baseado em tamanhos de fragmento codificados em hexadecimal.

> **Quando o Chunked Transfer é Ativado**
> O servidor omite o cabeçalho Content-Length e inclui Transfer-Encoding: chunked.
> O cliente HTTP detecta este cabeçalho e ativa o modo de leitura incremental.
> Em HTTP/2, o chunked encoding é transparente — a fragmentação é gerenciada pelo próprio protocolo.

2.2.1 Estrutura de Cada Chunk

Cada fragmento é composto pela concatenação de: o tamanho do payload em hexadecimal, um CRLF (\r\n), os dados do payload, e outro CRLF. O fim do stream é sinalizado por um chunk de tamanho zero, seguido de um CRLF final:

```
  HTTP Wire Format — Chunked Encoding
/* Estrutura geral de um chunk */
<TAMANHO-EM-HEX>\r\n
<DADOS>\r\n

/* Exemplo: palavra "Streaming" (9 bytes = 0x9) */
9\r\n
Streaming\r\n

/* Chunk terminador (fim do stream) */
0\r\n
\r\n
```

```
/* Resposta HTTP completa com dois chunks */
HTTP/1.1 200 OK
Transfer-Encoding: chunked
Content-Type: application/json
X-Accel-Buffering: no

7\r\n
{"id":1\r\n
8\r\n
,"val":5}\r\n
0\r\n
\r\n
```

2.2.2 O Papel do flush() no Servidor

A simples escrita no OutputStream do servidor não garante o envio imediato do chunk. O sistema operacional e o framework podem bufferizar os bytes para otimizar o desempenho de rede. Para forçar o envio imediato — essencial em streaming — é necessário invocar flush() após cada escrita:

```
  Importância do flush()
// Java: sem flush() os dados ficam presos no buffer interno
writer.write("chunk de dados");
writer.flush(); // obrigatório para emitir o chunk imediatamente

# Python/Flask: yield já faz flush implicitamente em cada iteração
def gerador():
    yield "chunk 1"   # enviado imediatamente
    yield "chunk 2"   # enviado imediatamente
```

2.3 Buffering Intermediário: O Principal Obstáculo

O maior problema prático do streaming HTTP é o buffering intermediário. Proxies reversos (Nginx, Apache), balanceadores de carga (HAProxy) e CDNs acumulam chunks antes de repassá-los ao cliente, anulando completamente o efeito do streaming. Para desabilitar este comportamento:

```
  Nginx — Configuração Anti-Buffering
# Nginx — desabilitar buffering para endpoints de streaming
location /api/streaming/ {
    proxy_pass         http://backend;
    proxy_buffering    off;
    proxy_cache        off;
    proxy_read_timeout 86400s;
    proxy_http_version 1.1;
    add_header         X-Accel-Buffering no;
}

# Cabeçalho HTTP alternativo (instruções ao proxy via resposta)
X-Accel-Buffering: no
```

2.4 NDJSON — Newline-Delimited JSON

Para streaming de objetos JSON, o formato NDJSON (application/x-ndjson) é amplamente utilizado. Cada linha do stream é um objeto JSON independente e completo, separado por \n. Esse formato permite que o cliente processe cada objeto ao recebê-lo, sem aguardar o fechamento de um array JSON:

```
  NDJSON vs JSON Array
/* NDJSON — cada linha é um JSON independente */
{"id":1,"nome":"Registro A","valor":42.0}
{"id":2,"nome":"Registro B","valor":17.5}
{"id":3,"nome":"Registro C","valor":98.1}

/* Diferente de JSON Array (não adequado para streaming) */
[{"id":1,...}, {"id":2,...}, {"id":3,...}]
/* ^ o cliente só processa após receber o "]" final */
```

3. Configuração do Ambiente

Esta atividade pode ser realizada com Java (Spring Boot) ou Python (Flask ou FastAPI). Escolha uma das opções abaixo e configure o ambiente conforme as instruções.

3.1 Opção A — Java (Spring Boot)

```
  Java — Setup
<!-- pom.xml: dependências mínimas -->
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
</dependencies>

# Criar projeto via Spring Initializr (https://start.spring.io)
# Grupo:    br.edu.pucgoias
# Artefato: chunked-streaming-lab
# Java:     17 ou 21
# Empacotamento: Jar

# Executar
./mvnw spring-boot:run
# Servidor disponível em http://localhost:8080
```

3.2 Opção B — Python

```
  Python — Setup
# Criar ambiente virtual
python3 -m venv venv
source venv/bin/activate    # Linux/macOS
venv\Scripts\activate       # Windows

# Flask
pip install flask

# FastAPI (alternativa)
pip install fastapi uvicorn

# Executar Flask
flask --app app run --debug
# Executar FastAPI
uvicorn app:app --reload
# Servidor disponível em http://localhost:5000 (Flask) ou :8000 (FastAPI)
```

3.3 Ferramentas de Suporte (Obrigatórias)

- Navegador com DevTools (Chrome/Firefox): aba Network para observar a resposta chunked em tempo real;
- curl: para consumir o endpoint diretamente no terminal e observar os dados chegando progressivamente;
- Postman ou Insomnia: opcional, para testes adicionais de endpoints HTTP;
- Git: para versionamento e entrega do código-fonte.

Verificando o Chunked Encoding com curl

Execute: curl -v http://localhost:8080/api/stream/registros

Na saída, procure pelo cabeçalho "< Transfer-Encoding: chunked" na resposta.

Os dados devem aparecer progressivamente no terminal enquanto o servidor ainda está processando.

4. Tarefas Práticas

1

Endpoint de Exportação de Registros via Chunked Streaming (2,5 pts)

Implemente um endpoint HTTP que transmita 5.000 registros simulados de um banco de dados em formato NDJSON utilizando Chunked Transfer Encoding. Cada registro deve conter um identificador sequencial, um nome gerado programaticamente e um valor numérico. O servidor não deve armazenar todos os registros em memória antes de iniciar a transmissão.

Requisitos funcionais:

- O endpoint deve responder em GET /api/stream/registros.
- O Content-Type da resposta deve ser application/x-ndjson.
- Cada linha transmitida deve ser um objeto JSON válido com os campos id (inteiro), nome (string) e valor (double com 2 casas decimais).
- O servidor deve adicionar um atraso de 1 ms entre cada registro para simular latência de banco de dados.
- O cabeçalho X-Accel-Buffering: no deve estar presente na resposta.

Esqueleto para Java (Spring Boot):

Java — Esqueleto

```
@RestController
@RequestMapping("/api/stream")
public class StreamingController {

    @GetMapping(value = "/registros", produces = "application/x-
ndjson")
    public ResponseEntity<StreamingResponseBody> streamRegistros() {

        StreamingResponseBody body = outputStream -> {
            // TODO: implementar o loop de 5.000 registros
            // Dica: use BufferedWriter + writer.flush() após cada
linha
        };

        return ResponseEntity.ok()
            .header("X-Accel-Buffering", "no")
            .contentType(MediaType.parseMediaType("application/x-
ndjson"))
            .body(body);
    }
}
```

Esqueleto para Python (Flask):

```
  Python — Esqueleto
from flask import Flask, Response
import json, time

app = Flask(__name__)

@app.route("/api/stream/registros")
def stream_registros():
    def gerar():
        # TODO: implementar o gerador de 5.000 registros
        # Dica: cada yield deve produzir uma linha NDJSON terminada
em "\n"
        pass

    return Response(
        gerar(),
        mimetype="application/x-ndjson",
        headers={"X-Accel-Buffering": "no"}
    )
```

2

Cliente Web com Fetch API e ReadableStream (2,5 pts)

Implemente uma página HTML com JavaScript que consuma o endpoint da Tarefa 1 utilizando a Fetch API com ReadableStream. A interface deve exibir os registros na tela progressivamente, à medida que chegam, sem aguardar a conclusão do stream.

Requisitos funcionais:

- Um botão "Iniciar Stream" deve disparar a requisição ao endpoint.
- Os registros devem ser exibidos em uma lista ou tabela HTML que cresce progressivamente.
- Um contador de registros recebidos deve ser atualizado em tempo real.
- Um indicador de status ("Conectando...", "Recebendo...", "Concluído") deve refletir o estado da conexão.
- Ao concluir o stream, o tempo total decorrido (em segundos) deve ser exibido.

Esqueleto JavaScript:

```
  JavaScript — Esqueleto ReadableStream
async function iniciarStream() {
    const response = await fetch("/api/stream/registros");
    const reader = response.body.getReader();
    const decoder = new TextDecoder();
    let buffer = "";
    let contador = 0;
```

```
    while (true) {
        const { done, value } = await reader.read();
        if (done) break;

        buffer += decoder.decode(value, { stream: true });

        // TODO: processar linhas completas no buffer
        // Dica: separar por "\n", manter a última linha incompleta no
buffer
        // Para cada linha completa: fazer JSON.parse() e exibir na tela
    }
}
```

Atenção: CORS e Proxy

Se o frontend for servido em uma porta diferente do backend (ex.: Live Server no VS Code), configure o CORS no servidor ou sirva a página HTML pelo próprio backend.

Em Flask: pip install flask-cors; from flask_cors import CORS; CORS(app).

3

Endpoint de Streaming de Texto Progressivo (Padrão LLM) (2,0 pts)

Implemente um segundo endpoint que simule o padrão de streaming de tokens adotado por modelos de linguagem (como ChatGPT e Claude). O servidor deve transmitir uma resposta de texto palavra por palavra, com um atraso de 100 ms entre cada palavra.

Requisitos funcionais:

- O endpoint deve responder em GET /api/stream/texto?prompt={texto}.
- O Content-Type deve ser text/plain; charset=utf-8.
- O servidor deve dividir o parâmetro prompt em palavras e retransmiti-las uma por vez com delay de 100 ms.
- Se o parâmetro prompt não for fornecido, utilize a frase padrão: "O streaming de dados permite a transmissão progressiva de informacoes sem buffering total na memoria do servidor."
- O cliente (página HTML da Tarefa 2) deve exibir o texto sendo construído palavra por palavra, como um efeito de digitação.

Exemplo de uso com curl:

```
  curl — Teste do Endpoint
# Enviar prompt via query parameter
curl -N "http://localhost:8080/api/stream/texto?prompt=Ola+mundo"
```

```
# Saída esperada (com delays visíveis entre palavras):
Ola
mundo

# Flag -N desabilita o buffering do curl
```

Extensão Opcional (+0,5 ponto extra)

Implemente o endpoint /api/stream/texto aceitando POST com corpo JSON no formato {"prompt": "sua pergunta aqui"} e retorne o texto palavra por palavra via chunked.

Adapte o cliente JavaScript para enviar o prompt via POST fetch com method: "POST".

4

Análise de Rede e Relatório Técnico (3,0 pts)

Utilizando as ferramentas de desenvolvedor do navegador (aba Network) e o curl, analise o comportamento da rede durante a execução dos endpoints implementados nas Tarefas 1 e 2. Documente as observações em um relatório técnico conforme as instruções abaixo.

Questões a responder no relatório:

4.1 Análise do Cabeçalho HTTP (0,5 pt)

Capture os cabeçalhos completos da resposta do endpoint /api/stream/registros utilizando curl -v e cole no relatório. Identifique e explique:

- O cabeçalho que confirma o modo chunked encoding;
- A ausência do Content-Length e sua justificativa técnica;
- O papel do cabeçalho X-Accel-Buffering: no.

4.2 Medição de Latência Percebida (0,5 pt)

Implemente uma versão não-streaming do endpoint de exportação que retorne todos os 5.000 registros em um único array JSON. Compare os tempos de:

- Time to First Byte (TTFB) na versão com streaming vs. sem streaming (observe no DevTools > Network > Timing);
- Tempo total de carregamento nas duas versões;
- Apresente os valores em uma tabela comparativa e interprete os resultados.

4.3 Experimento de Buffering (1,0 pt)

Remova o cabeçalho X-Accel-Buffering: no e o flush() (ou o yield incremental em Python) da implementação e observe o comportamento na aba Network do DevTools. Documente:

- O que acontece com o TTFB e com a progressividade da exibição no frontend;
- Explique por que o buffering ocorre mesmo com Transfer-Encoding: chunked no cabeçalho;
- Restaure a implementação correta e documente a diferença observada.

4.4 Questões Conceituais (1,0 pt)

Responda objetivamente no relatório:

a) Em qual versão do protocolo HTTP o Chunked Transfer Encoding é suportado nativamente? Como o HTTP/2 trata a fragmentação de dados de forma diferente?

b) Cite dois cenários de aplicação reais (diferentes dos implementados nesta atividade) em que o Chunked Transfer Encoding seria a escolha mais adequada em comparação ao modelo request-response convencional. Justifique.

c) Um colega afirma que "basta usar Transfer-Encoding: chunked para ter streaming verdadeiro." Essa afirmação está correta? Explique os fatores que podem invalidá-la na prática.

d) Qual é a diferença entre NDJSON e um array JSON convencional no contexto de streaming? Por que o NDJSON é mais adequado para transmissão progressiva?

5. Formato de Entrega

A submissão deve ser feita via Moodle no prazo estipulado, contendo um único arquivo .zip com a seguinte estrutura:

```
  Estrutura do .zip
AT02_ChunkedStreaming_<NomeCompleto>/
├── codigo/
│   ├── src/                        (Java: código-fonte Spring Boot)
│   │   └── ...                     (ou)
│   ├── app.py                      (Python: arquivo principal Flask/FastAPI)
│   ├── static/
│   │   └── index.html              (página HTML do cliente)
│   └── requirements.txt / pom.xml
└── relatorio/
    └── AT02_Relatorio_<Nome>.pdf   (relatório das análises da Tarefa 4)
```

Requisitos Obrigatórios para Correção

O projeto deve executar sem erros com os comandos de setup informados na Seção 3.

O arquivo relatorio/AT02_Relatorio_<Nome>.pdf deve estar presente e conter as respostas de todas as sub-questões da Tarefa 4.

Trabalhos sem o relatório perderão automaticamente os 3,0 pontos da Tarefa 4.

Plágio (código ou relatório idêntico entre grupos) resulta em nota zero para todos os envolvidos.

5.1 Critérios de Avaliação

Critério de Avaliação	Descrição / Evidências Esperadas	Pontos
Tarefa 1 — Endpoint NDJSON	Endpoint funcional em GET /api/stream/registros. Stream com 5.000 registros NDJSON. Flush incremental ativo. Cabeçalhos corretos (Content-Type, X-Accel-Buffering). Delay de 1ms entre registros.	**2.5**
Tarefa 2 — Cliente Fetch API	Página HTML funcional. Exibição progressiva de registros. Contador em tempo real. Indicador de status. Tempo decorrido ao finalizar. Processamento correto do buffer de linhas parciais.	**2.5**
Tarefa 3 — Streaming de Texto	Endpoint GET /api/stream/texto funcional. Transmissão palavra por palavra com delay de 100ms. Integração visual no frontend (efeito de digitação). Texto padrão quando sem prompt.	**2**
Tarefa 4.1 — Análise de Cabeçalhos	Captura dos cabeçalhos via curl -v. Identificação correta do Transfer-Encoding: chunked, ausência do Content-Length e papel do X-Accel-Buffering.	**0.5**

Critério de Avaliação	Descrição / Evidências Esperadas	Pontos
Tarefa 4.2 — Medição de Latência	Implementação da versão não-streaming. Tabela comparativa com TTFB e tempo total. Interpretação correta dos resultados.	**0.5**
Tarefa 4.3 — Experimento Buffering	Documentação do comportamento com buffering habilitado. Explicação técnica da causa. Comparação before/after com evidências (screenshots ou medições).	**1**
Tarefa 4.4 — Questões Conceituais	Respostas objetivas e tecnicamente precisas às 4 questões. Pontuação parcial por questão (2,5 pts cada).	**1**
Qualidade do Código	Critério qualitativo (sem pontuação separada): código organizado, comentado e sem redundâncias contribui para a nota total.	**0**
	TOTAL	**10**

Nota: os pontos acima estão em escala de 0 a 10. Pontuação parcial é aplicada quando o critério é atendido de forma incompleta.